

Lab 2 report

2026/5/2
Xue Zhengyang

1. Introduction

This report summarizes a series of hands-on experiments designed to establish a foundational understanding of Docker containerization technology. The practical exercises covered five core areas. First, publishing container ports via the Docker CLI. Second, sharing files between host and container using bind mounts. Third, persisting data across container lifecycles using Docker volumes. Fourth, building and orchestrating a multi-container application with Nginx, Node.js, and Redis using both imperative docker run commands and declarative Docker Compose. Fifth, overriding container defaults including networking configurations, resource limits, and entrypoint commands. Through these exercises, the theoretical abstraction of containers was translated into concrete operational skills.

2. Experimental Setup

The experiments were conducted on a Windows host machine running Docker Desktop. The primary interface for command execution was Windows PowerShell. Prerequisites included the installation of Docker Desktop with WSL2 integration enabled, as well as Git for repository cloning. Key images used during the experiments included docker/welcome-to-docker, httpd:2.4, postgres:18, redis, and custom-built images for a Node.js web application and an Nginx reverse proxy.

3. Experiment 1: Container Port Publishing with Docker CLI

Objective: Understand how to expose container services to the host machine through port mapping.

Procedure: A container was launched using the command `docker run -d -p 8080:80 docker/welcome-to-docker`. The syntax `HOST_PORT:CONTAINER_PORT` was analyzed, where port 8080 on the localhost was bound to port 80 inside the container.

Observation: The application became accessible via `http://localhost:8080`. The Docker Desktop Dashboard visually confirmed the published port mapping.

4. Experiment 2: Sharing Data with Bind Mounts

Objective: Implement real-time file synchronization between the host filesystem and a container using bind mounts.

Procedure: An initial `httpd:2.4` container was started and subsequently removed. A local directory named `public_html` was created, containing a custom `index.html` file with a Whalecome theme. A new container was then launched with a bind mount using the command `docker run -d --name my_site -p 8080:80 -v ${PWD}:/usr/local/apache2/htdocs/`

httpd:2.4. On Windows PowerShell, the absolute path `${PWD}` was required instead of the relative `./` notation commonly used in Unix-like shells.

Observation: The custom webpage rendered immediately at `http://localhost:8080`. When `index.html` was deleted from the host, the file simultaneously disappeared inside the container, which was verified via the Docker Desktop Dashboard Files tab. Apache then returned an Index of / listing. Recreating the file on the host restored the site instantly.

Key Insight: Bind mounts create a live, bidirectional link between host and container, making them ideal for development environments where code changes must be reflected immediately.

5. Experiment 3: Persisting Data with Docker Volumes

Objective: Demonstrate data persistence independent of container lifecycle using named volumes.

Procedure: A PostgreSQL container was initialized with a named volume using the command `docker run --name=db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/postgresql/data postgres:18`. A table named `tasks` was created, and two records were inserted. The container was then stopped and removed. A new container named `new-db` was started, attaching the same `postgres_data` volume, but omitting the `POSTGRES_PASSWORD` environment variable.

Observation: The data remained intact. Querying the new container with `SELECT * FROM tasks;` returned the previously inserted records. The volume was later inspected and removed using `docker volume rm postgres_data` and `docker volume prune`.

Key Insight: Unlike bind mounts, volumes are managed by Docker and are decoupled from the host filesystem structure. They are the preferred mechanism for database persistence and stateful applications.

6. Experiment 4: Multi-Container Orchestration with Docker Run

Objective: Manually wire a multi-service application consisting of a Redis database, two Node.js backend instances, and an Nginx reverse proxy.

Procedure: After cloning the `nginx-node-redis` sample repository, custom images were built for the web and nginx services. A dedicated bridge network named `sample-app` was created to facilitate DNS-based service discovery. Containers were launched sequentially. First, the `redis` container with a network alias. Second, `web1` and `web2` with hostnames and aliases. Third, the `nginx` container exposing port 80 and routing traffic to the backend containers.

Observation: Accessing `http://localhost` showed a visit counter that incremented on each refresh. The hostname alternated between `web1` and `web2`, confirming that Nginx was distributing requests in a round-robin fashion. Both backends shared the same Redis instance, ensuring a global, consistent counter state.

Challenge: Port 80 was initially occupied by a pre-existing XAMPP Apache service, causing a conflict. Stopping the XAMPP service resolved the issue.

7. Experiment 5: Streamlining Deployment with Docker Compose

Objective: Replace manual, imperative container orchestration with a declarative compose.yml configuration.

Procedure: Navigating to the project root, the existing compose.yml file was used to define the entire stack. The application was launched with a single command: `docker compose up -d --build`.

Observation: A single command created the default network and started all four services. The behavior was identical to the manual setup, but the operational complexity was drastically reduced. Cleanup was similarly simplified using `docker compose down`.

Key Insight: Docker Compose transforms multi-container deployment from an error-prone sequence of commands into a version-controlled, reproducible infrastructure specification. This aligns with Infrastructure as Code principles.

8. Experiment 6: Overriding Container Defaults and Resource Management

Objective: Explore advanced docker run flags for network isolation, resource constraints, and command overrides.

Procedure: Several independent tests were conducted. First, two PostgreSQL containers were run simultaneously on host ports 5432 and 5433, both mapping internally to container port 5432. Second, a user-defined bridge network named mynetwork was created. Unlike the default bridge network, custom networks provide automatic DNS resolution by container name and better isolation. Third, a container was constrained using `--memory="512m"` and `--cpus=".5"` to prevent resource starvation on shared systems. Fourth, both docker run and compose.yml were used to override the default PostgreSQL startup behavior, specifying custom host and port parameters.

Observation: Containers on the custom network could resolve each other by name rather than IP address. Resource-limited containers operated within their constraints without affecting host stability.

9. Challenges and Troubleshooting

Several challenges were encountered during the experiments.

The Git clone operation initially failed with an RPC error and connection reset. This was caused by network instability or firewall interference. The issue was resolved by using a shallow clone with `git clone --depth 1` or by downloading the ZIP archive directly from GitHub.

When attempting to create the public_html directory, a permission denied error occurred because the command was executed in `C:\WINDOWS\system32`, which is a protected system directory. The solution was to navigate to the user home directory using `cd ~` before creating files.

A path error occurred when using the `cd` command with a username containing a space. The path `C:\Users\Xue Zhengyang` was parsed incorrectly by PowerShell. The solution was to use quoted paths such as `cd "C:\Users\Xue Zhengyang"`.

During the multi-container experiment, the browser displayed the XAMPP welcome page instead of the Docker application. This happened because the XAMPP Apache service was already running and occupying port 80. The conflict was resolved by stopping the Apache service through the XAMPP Control Panel.

Finally, PowerShell has a built-in alias that maps curl to Invoke-WebRequest, which behaves differently from the standard curl utility. This caused URI parsing errors. The solution was to use curl.exe with the explicit protocol prefix <http://>.

10. Key Takeaways and Reflections

The Ephemeral Nature of Containers vs. The Necessity of Persistence

The most profound realization from this series was the distinction between a container's ephemeral filesystem and persistent storage mechanisms. Without explicit volumes or bind mounts, container data vanishes upon deletion. This makes Docker volumes indispensable for stateful services like databases, while bind mounts excel in development workflows requiring rapid iteration.

Imperative vs. Declarative Orchestration

Running the Nginx-Node-Redis stack manually with six separate commands including docker network create, four docker run commands, and verification steps was tedious and brittle. A single typo in a network alias or port mapping would break the entire stack. Docker Compose eliminated this fragility by centralizing configuration. This experience solidified my appreciation for declarative tooling in modern DevOps practices.

Networking as a First-Class Concern

The default bridge network's lack of DNS resolution was a subtle but critical discovery. In real-world microservices architectures, services must discover each other dynamically. Custom bridge networks provide this capability natively, reinforcing the principle that container networking should never be an afterthought.

Cross-Platform Nuances

Working in Windows PowerShell revealed friction points that are invisible in Unix environments. These included the need for absolute paths in bind mounts, the space-character issue in file paths, and the curl alias conflict. These challenges underscored that containerization abstracts the runtime environment, but the host environment still matters during development and debugging.

Resource Governance

The resource-limiting exercise using --memory and --cpus served as a reminder that containers, while isolated, share the host kernel. On multi-tenant or production systems, unconstrained containers pose a stability risk. Proactive resource management is not merely an optimization but an operational requirement.

The Power of Layered Abstraction

From a single docker run -p command to a multi-service compose.yml, the experiments traced a clear trajectory of increasing abstraction. Yet each layer builds upon the previous one. Understanding docker run is a prerequisite to debugging Compose failures. This reinforces the value of mastering fundamentals before adopting higher-level tools.

11. Conclusion

This comprehensive Docker laboratory provided end-to-end exposure to container lifecycle management, storage strategies, networking, and orchestration. The progression from isolated containers to a load-balanced, multi-service architecture, first managed manually and then via Docker Compose, mirrors the evolution of container usage in industry settings. The hands-on troubleshooting of port conflicts, path syntax issues, and network behavior transformed abstract documentation into operational intuition. Moving forward, these skills form the foundation for more advanced topics, including container security, CI/CD pipeline integration, and Kubernetes orchestration.

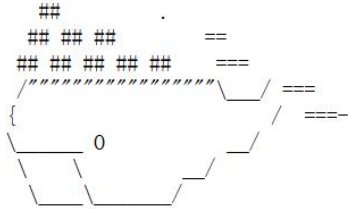
Experiment Screenshot:

```
PS C:\Users\Xue Zhengyang> cd public_html
PS C:\Users\Xue Zhengyang\public_html> @'
>> <!DOCTYPE html>
>> <html lang="en">
>> <head>
>> <meta charset="UTF-8">
>> <title>My Website with a Whale & Docker!</title>
>> </head>
>> <body>
>> <h1>Whalecome!!</h1>
>> <p>Look! There's a friendly whale greeting you!</p>
>> <pre id="docker-art">
>>     ##
>>     ## ## ##      ==
>>     ## ## ## ## ##   ===
>>     /*****\_____/ ===
>>     {                      /  ===-
>>     \_____ 0      _--/
>>     \_____/_--/
>>
>> Hello from Docker!
>> </pre>
>> </body>
>> </html>
>> '@ | Out-File -FilePath index.html -Encoding utf8
PS C:\Users\Xue Zhengyang\public_html> |
cba947608522 docker.gitea.com/gitea:1.26.1 "/usr/bin/entrypoint..." 5 days ago
Up 26 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/t
cp, [::]:222->22/tcp gitea
PS C:\Users\Xue Zhengyang\public_html> docker rm -f my_site
my_site
PS C:\Users\Xue Zhengyang\public_html> cd ~
PS C:\Users\Xue Zhengyang> cd public_html
PS C:\Users\Xue Zhengyang\public_html> docker run -d --name my_site -p 8080:80 -v ${PWD}
}:/usr/local/apache2/htdocs/ httpd:2.4
8eda5f8966571a9ce1e761e2531c0c91a84b82179c6da1d9b0933add4f8d331a
PS C:\Users\Xue Zhengyang\public_html> docker exec my_site ls /usr/local/apache2/htdocs
/
index.html
PS C:\Users\Xue Zhengyang\public_html> docker exec my_site cat /usr/local/apache2/htdoc
s/index.html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My Website with a Whale & Docker!</title>
</head>
<body>
<h1>Whalecome!!</h1>
<p>Look! There's a friendly whale greeting you!</p>
<pre id="docker-art">
    ##
    ## ## ##      ==
    ## ## ## ## ##   ===
    /*****\_____/ ===
    {                      /  ===-
    \_____ 0      _--/
    \_____/_--/

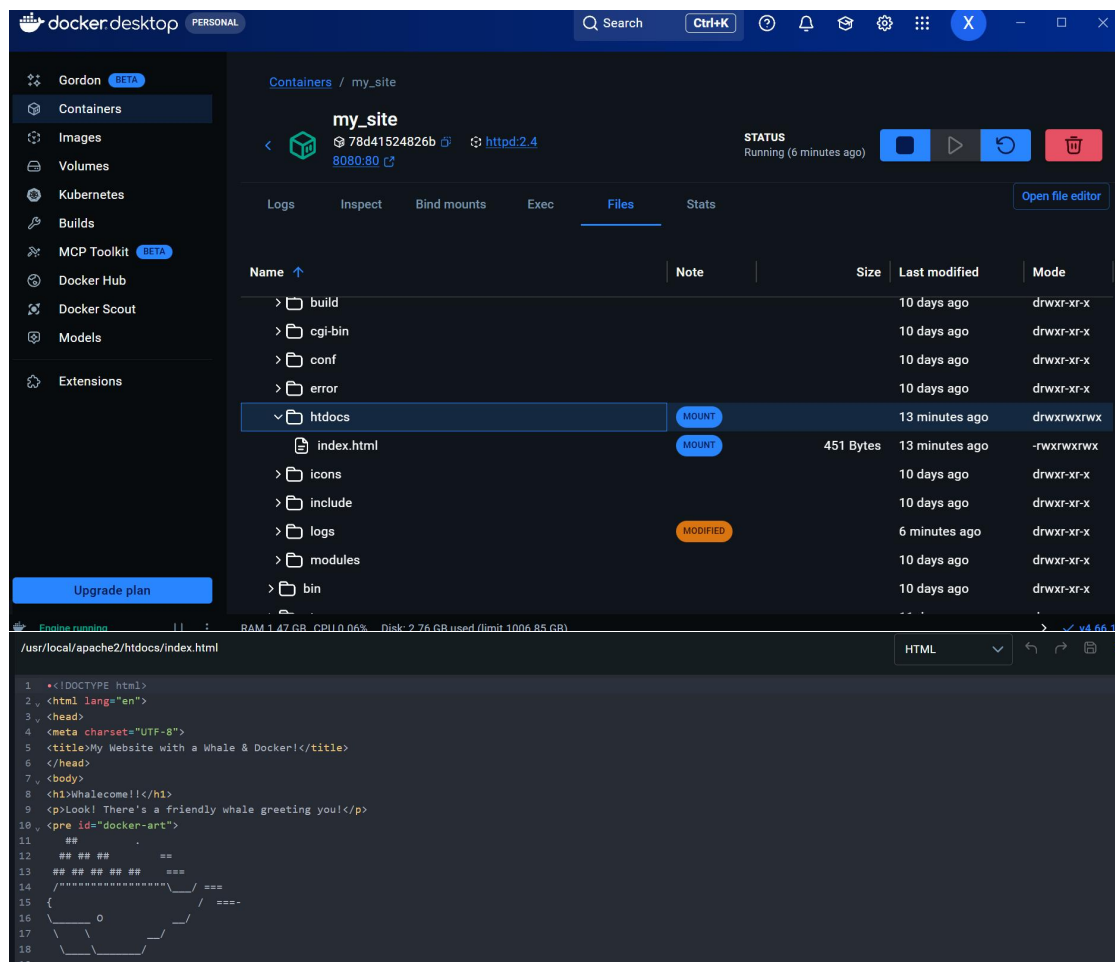
Hello from Docker!
</pre>
</body>
</html>
PS C:\Users\Xue Zhengyang\public_html> docker rm -f my_site
my_site
PS C:\Users\Xue Zhengyang\public_html> cd "$env:USERPROFILE\public_html"
PS C:\Users\Xue Zhengyang\public_html> docker run -d --name my_site -p 8080:80 -v "${PW
D}:/usr/local/apache2/htdocs/" httpd:2.4
78d41524826b283185a49084bdae34f659ddbba1d2cf8ac5fffb9b8ab6b674a7
PS C:\Users\Xue Zhengyang\public_html>
```

Whalecome!!

Look! There's a friendly whale greeting you!

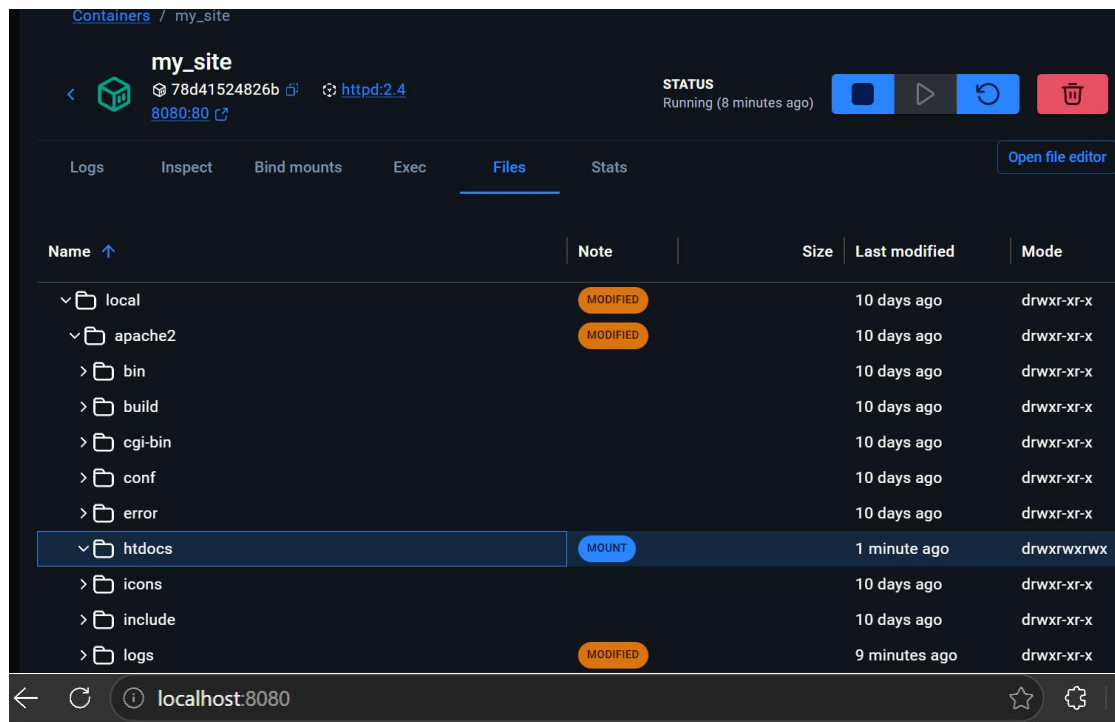


Hello from Docker!



The screenshot shows the Docker Desktop application window. On the left is a sidebar with navigation options: Containers, Images, Volumes, Kubernetes, Builds, MCP Toolkit (BETA), Docker Hub, Docker Scout, Models, and Extensions. The main area is titled 'Containers / my_site'. It shows the container 'my_site' with ID '78d41524826b' and port 'http://2.4.8080.80'. The status is 'Running (6 minutes ago)'. Below this are tabs for Logs, Inspect, Bind mounts, Exec, Files, and Stats. The 'Files' tab is selected, displaying a file browser view of the container's filesystem. The files listed are: build, cgi-bin, conf, error, htdocs (expanded), index.html (451 Bytes), icons, include, logs, modules, and bin. The 'htdocs' directory is expanded, showing 'index.html' which is marked as 'MOUNTED'. The bottom pane shows the content of 'index.html' in a code editor, displaying HTML code that includes the 'Whalecome!!' message and the ASCII art whale.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <title>My Website with a Whale & Docker!</title>
6 </head>
7 <body>
8 <h1>Whalecome!!</h1>
9 <p>Look! There's a friendly whale greeting you!</p>
10 <pre id="docker-art">
11     ##
12     ## ## ##
13     ## ## ## ##
14     /===== \
15     {
16     0
17     }
18     \===== \
19     ##
```



Index of /

```
PS C:\Users\Xue Zhengyang\public_html> cd ~
PS C:\Users\Xue Zhengyang> cd public_html
PS C:\Users\Xue Zhengyang\public_html> Remove-Item index.html
PS C:\Users\Xue Zhengyang\public_html> '@'
>> <!DOCTYPE html>
>> <html>
>> <head><title>I'm back!</title></head>
>> <body><h1>The whale is back!</h1></body>
>> </html>
>> '@ | Out-File -FilePath index.html -Encoding utf8
PS C:\Users\Xue Zhengyang\public_html> |
```

Containers / my_site

my_site

78d41524826b <http://8080:80> STATUS Running (8 minutes ago)

Logs Inspect Bind mounts Exec **Files** Stats [Open file editor](#)

Name ↑	Note	Size	Last modified	Mode
✓ local	MODIFIED		10 days ago	drwxr-xr-x
✓ apache2	MODIFIED		10 days ago	drwxr-xr-x
> bin			10 days ago	drwxr-xr-x
> build			10 days ago	drwxr-xr-x
> cgi-bin			10 days ago	drwxr-xr-x
> conf			10 days ago	drwxr-xr-x
> error			10 days ago	drwxr-xr-x
✓ htdocs	MOUNT		5 minutes ago	drwxrwxrwx
index.html	MOUNT	114 Bytes	22 seconds ago	-rwxrwxrwx
> icons			10 days ago	drwxr-xr-x
> include			10 days ago	drwxr-xr-x

RAM 1.46 GB CPU 0.06% Disk: 2.76 GB used (limit 1006.85 GB) >_ ✓ v4.66.1

The whale is back!

```
PS C:\Users\Xue Zhengyang\public_html> cd ~
PS C:\Users\Xue Zhengyang> cd public_html
PS C:\Users\Xue Zhengyang\public_html> Remove-Item index.html
PS C:\Users\Xue Zhengyang\public_html> @'
>> <!DOCTYPE html>
>> <html>
>> <head><title>I'm back!</title></head>
>> <body><h1>The whale is back!</h1></body>
>> </html>
>> '@ | Out-File -FilePath index.html -Encoding utf8
PS C:\Users\Xue Zhengyang\public_html> docker stop my_site
my_site
PS C:\Users\Xue Zhengyang\public_html>

PS C:\Users\Xue Zhengyang> git clone https://github.com/dockeramples/nginx-node-redis
Cloning into 'nginx-node-redis'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (10/10), done.
Receiving objects: 59% (49/82) used 18 (delta 18), pack-reused 54 (from 1)
Receiving objects: 100% (82/82), 76.62 KiB | 1.22 MiB/s, done.
Resolving deltas: 100% (26/26), done.
```

```
PS C:\Users\Xue Zhengyang\nginx-node-redis> Get-ChildItem
```

目录: C:\Users\Xue Zhengyang\nginx-node-redis

Mode	LastWriteTime	Length	Name
d-----	2026/5/2 11:30		nginx
d-----	2026/5/2 11:30		web
-a----	2026/5/2 11:30	378	compose.yml
-a----	2026/5/2 11:30	181	CONTRIBUTING.md
-a----	2026/5/2 11:30	11558	LICENSE
-a----	2026/5/2 11:30	3632	README.md

```
PS C:\Users\Xue Zhengyang\nginx-node-redis> cd nginx
```

```
PS C:\Users\Xue Zhengyang\nginx-node-redis\nginx> docker build -t nginx .
```

```
[+] Building 15.0s (9/9) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 141B                                0.0s
=> [internal] load metadata for docker.io/library/nginx:1.29      5.0s
=> [auth] library/nginx:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                       0.0s
=> [1/3] FROM docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455 8.4s
=> => resolve docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455 0.1s
=> => sha256:677c631968686eeb23ab8dd436d49bde041266df5d8952f03d 1.21kB / 1.21kB 0.7s
=> => sha256:801a1ad15b4e00add388aca409568400fb8071019d6ba83995 1.40kB / 1.40kB 1.1s
=> => sha256:ff048f1f2159a060f69b1861ea262b839cc6e77a9389848929f702 403B / 403B 1.3s
=> => sha256:4677c2a9a3d4f9290cb784d95a9e16378655ecdd7df9e77668d391 956B / 956B 1.5s
=> => sha256:85c66128325abc04138f6944d943e5279375665f6dbefe7f4f6b5e 629B / 629B 0.8s
=> => sha256:ce776bbcdad0d6bf4da8df324b82066a03f45bfbbbe520df5 33.16MB / 33.16MB 6.2s
=> => extracting sha256:ce776bbcdad0d6bf4da8df324b82066a03f45bfbbbe520df535293ae 0.5s
=> => extracting sha256:85c66128325abc04138f6944d943e5279375665f6dbefe7f4f6b5e9 0.0s
=> => extracting sha256:4677c2a9a3d4f9290cb784d95a9e16378655ecdd7df9e77668d3915 0.0s
=> => extracting sha256:ff048f1f2159a060f69b1861ea262b839cc6e77a9389848929f7027 0.1s
=> => extracting sha256:677c631968686eeb23ab8dd436d49bde041266df5d8952f03d7a8c4 0.0s
=> => extracting sha256:801a1ad15b4e00add388aca409568400fb8071019d6ba83995f4317 0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 222B                                      0.0s
=> [2/3] RUN rm /etc/nginx/conf.d/default.conf                  0.6s
=> [3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf         0.1s
=> exporting to image                                           0.5s
=> => exporting layers                                              0.2s
=> => exporting manifest sha256:5673b480455c4299a715051ddc0722ad7e34f5a59dbb3da 0.0s
=> => exporting config sha256:a794cc54c7ebf86c875a1aa8a7354016a49f768d9eae92f8a 0.0s
=> => exporting attestation manifest sha256:b0069c696158ece6e472e41a6e91a9ff9c6 0.0s
=> => exporting manifest list sha256:3d88ae599d325e69ae79067233cb921c38a345ba41 0.0s
=> => naming to docker.io/library/nginx:latest                    0.0s
=> => unpacking to docker.io/library/nginx:latest                  0.1s
PS C:\Users\Xue Zhengyang\nginx-node-redis\nginx> |
```

```

PS C:\Users\Xue Zhengyang\nginx-node-redis\nginx> cd ..
PS C:\Users\Xue Zhengyang\nginx-node-redis> cd web
PS C:\Users\Xue Zhengyang\nginx-node-redis\web> docker build -t web .
[+] Building 79.8s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 175B                                0.0s
=> [internal] load metadata for docker.io/library/node:21         5.2s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                      0.0s
=> [1/5] FROM docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279ef 69.5s
=> => resolve docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279efa 0.1s
=> => sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebe 449B / 449B 0.7s
=> => sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2 1.25MB / 1.25MB 1.4s
=> => sha256:4684276225df21ffeac91884cd316f7bc0d99baeef8ff81 49.72MB / 49.72MB 27.1s
=> => sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8 3.37kB / 3.37kB 1.4s
=> => sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20a 211.21MB / 211.21MB 63.6s
=> => sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d 64.14MB / 64.14MB 37.8s
=> => sha256:891494355808bdd3db5552f0d3723fd0fa826675f774853 24.05MB / 24.05MB 16.3s
=> => sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549 49.58MB / 49.58MB 25.0s
=> => extracting sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549687b41df 1.1s
=> => extracting sha256:891494355808bdd3db5552f0d3723fd0fa826675f774853796faf2 0.5s
=> => extracting sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d2d81c4f8 1.4s
=> => extracting sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20ac0584db70c 3.5s
=> => extracting sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8c8c5a 0.0s
=> => extracting sha256:4684276225df21ffeac91884cd316f7bc0d99baeef8ff811f98c2ab 1.2s
=> => extracting sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2b043d 0.1s
=> => extracting sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebef 0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 17.30kB                                0.0s
=> [2/5] WORKDIR /usr/src/app                                    0.4s
=> [3/5] COPY package.json package-lock.json ./                  0.1s
=> [4/5] RUN npm ci                                              3.2s
=> [5/5] COPY ./server.js ./                                     0.1s
=> exporting to image                                             0.9s
=> => exporting layers                                             0.4s
=> => exporting manifest sha256:afbfff4e2bf162000dc8f02b27030b189d1eabc08aad4e65 0.0s
=> => exporting config sha256:ac97e78e7c6c0a26aba1e0cc1df782ae5ccbb943be11dc5 0.0s
=> => exporting attestation manifest sha256:f6645dd79c6a7c3b0919fbeb1353b2c6ab 0.1s
=> => exporting manifest list sha256:e3fb2b726857be2af59973555f8c4652f0bb39a611 0.0s
=> => naming to docker.io/library/web:latest                       0.0s
=> => unpacking to docker.io/library/web:latest                     0.3s
PS C:\Users\Xue Zhengyang\nginx-node-redis\web> cd ..
PS C:\Users\Xue Zhengyang\nginx-node-redis>

PS C:\Users\Xue Zhengyang\nginx-node-redis> docker network create sample-app
8c93a556015d9a8315284447ade7c8a2af310c85fe0b70585537288a7049c25a
PS C:\Users\Xue Zhengyang\nginx-node-redis> docker run -d --name redis --network sample-app --network-alias redis redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
d55df8e0f53d: Pull complete
4f4fb700ef54: Pull complete
2b8951cd65b5: Pull complete
9e0f42ff49e1: Pull complete
14d3b17b0010: Pull complete
37afe6025aa9: Pull complete
f426aaaf8949: Download complete
ff0fc1b2ba98: Download complete
Digest: sha256:832d7785830f3f4b559300e6191fc914b15642c1935252338825cf4332200148
Status: Downloaded newer image for redis:latest
585efe55575c07c965750ca99b090ac606b65b3bedbce96dee68f7a722a2beb7
PS C:\Users\Xue Zhengyang\nginx-node-redis>

```

```

PS C:\Users\Xue Zhengyang\nginx-node-redis> docker run -d --name web1 -h web1 --network
sample-app --network-alias web1 web
ac078b55c8b84c67fc600d01b8bbafeaabd4120055f582382e2f0085f9c5b483
PS C:\Users\Xue Zhengyang\nginx-node-redis> docker run -d --name web2 -h web2 --network
sample-app --network-alias web2 web
4671cf9a4482afaf2487585aa9179b2e5f441bb33970e336860f4ff4877a8aa3
PS C:\Users\Xue Zhengyang\nginx-node-redis> docker run -d --name nginx --network sample
-app -p 80:80 nginx
f4727690aa1248039b3801fc9da49df24c96cefaaa1b906641197a53a52ba102
PS C:\Users\Xue Zhengyang\nginx-node-redis> docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED
f4727690aa12	nginx	"/docker-entrypoint..."	8 seconds ago
Up 7 seconds	0.0.0.0:80->80/tcp, [::]:80->80/tcp		
4671cf9a4482	web	"docker-entrypoint.s..."	14 seconds ago
Up 13 seconds	web2		
ac078b55c8b8	web	"docker-entrypoint.s..."	22 seconds ago
Up 21 seconds	web1		
585efe55575c	redis	"docker-entrypoint.s..."	44 seconds ago
Up 43 seconds	6379/tcp		
cba947608522	docker.gitea.com/gitea:1.26.1	"/usr/bin/entrypoint..."	5 days ago
Up 58 minutes	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp	gitea	

```

PS C:\Users\Xue Zhengyang\nginx-node-redis> |

```

← ↻ ⓘ localhost

web1: Number of visits is: 15

```
PS C:\Users\Xue Zhengyang\nginx-node-redis> cd ~
PS C:\Users\Xue Zhengyang> cd nginx-node-redis
PS C:\Users\Xue Zhengyang\nginx-node-redis> Get-ChildItem
```

目录: C:\Users\Xue Zhengyang\nginx-node-redis

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	2026/5/2	11:30		nginx
d-----	2026/5/2	11:30		web
-a----	2026/5/2	11:30	378	compose.yml
-a----	2026/5/2	11:30	181	CONTRIBUTING.md
-a----	2026/5/2	11:30	11558	LICENSE
-a----	2026/5/2	11:30	3632	README.md

```
PS C:\Users\Xue Zhengyang\nginx-node-redis> Get-Content compose.yml
```

```
services:
  redis:
    image: redis
    ports:
      - '6379:6379'
  web1:
    restart: on-failure
    build: ./web
    hostname: web1
    ports:
      - '81:5000'
  web2:
    restart: on-failure
    build: ./web
    hostname: web2
    ports:
      - '82:5000'
  nginx:
    build: ./nginx
    ports:
      - '80:80'
    depends_on:
      - web1
      - web2
```

```
PS C:\Users\Xue Zhengyang\nginx-node-redis> |
```

```

PS C:\Users\Xue Zhengyang\nginx-node-redis> docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.50kB 0.0s done
#1 DONE 0.0s

#2 [web1 internal] load build definition from Dockerfile
#2 transferring dockerfile: 175B 0.0s done
#2 DONE 0.0s

#3 [nginx internal] load build definition from Dockerfile
#3 transferring dockerfile: 141B done
#3 DONE 0.0s

#4 [web2 internal] load metadata for docker.io/library/node:21
#4 ...

#5 [auth] library/nginx:pull token for registry-1.docker.io
#5 DONE 0.0s

#6 [nginx internal] load metadata for docker.io/library/nginx:1.29
#6 ...

#7 [auth] library/node:pull token for registry-1.docker.io
#7 DONE 0.0s

#6 [nginx internal] load metadata for docker.io/library/nginx:1.29
#6 DONE 2.0s

#4 [web2 internal] load metadata for docker.io/library/node:21
#4 ...

#8 [nginx internal] load .dockerignore
#8 transferring context: 2B done
#8 DONE 0.0s

#9 [nginx internal] load build context
#9 transferring context: 32B done
#9 DONE 0.0s

#10 [nginx 1/3] FROM docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
#10 resolve docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc 0.0s done

#22 [web2] exporting to image
#22 exporting layers done
#22 exporting manifest sha256:308bf0cc0bc4f98859ffd08611d34076611b244f37eaf99be5c1cec07f710e6a 0.1s done
#22 exporting config sha256:b2077a89045fb26671d4484a26ff3f780734a9ff87a8972ae8ce3b81d68e405e 0.1s done
#22 exporting attestation manifest sha256:8f58168dc6f2b2ccc350d15f940f38708564444a3fca51e5e4923a483f6fdf5d 0.1s done
#22 exporting manifest list sha256:1b2c837e043c41adab4a70bcade6b8e5c97aaae3f5b02eea3303d98c87d29357 0.1s done
#22 naming to docker.io/library/nginx-node-redis-web2:latest 0.0s done
#22 unpacking to docker.io/library/nginx-node-redis-web2:latest 0.0s done
#22 DONE 0.5s

#23 [nginx] resolving provenance for metadata file
#23 DONE 0.0s

#24 [web1] resolving provenance for metadata file
#24 DONE 0.1s

#25 [web2] resolving provenance for metadata file
#25 DONE 0.0s
[+] up 8/8
✔Image nginx-node-redis-web2 Built 4.4s
✔Image nginx-node-redis-nginx Built 4.4s
✔Image nginx-node-redis-web1 Built 4.4s
>✔Network nginx-node-redis_default Created 0.1s
✔Container nginx-node-redis-redis-1 Started 0.7s
✔Container nginx-node-redis-web2-1 Started 0.8s
✔Container nginx-node-redis-web1-1 Started 0.8s
✔Container nginx-node-redis-nginx-1 Started 1.0s
PS C:\Users\Xue Zhengyang\nginx-node-redis> |

```

```
web1: Number of visits is: 1
```

```
PS C:\Users\Xue Zhengyang\nginx-node-redis> docker compose down
[+] down 5/5
```

✓Container	nginx-node-redis-nginx-1	Removed	0.6
✓Container	nginx-node-redis-redis-1	Removed	0.5
✓Container	nginx-node-redis-web2-1	Removed	0.9
✓Container	nginx-node-redis-web1-1	Removed	1.0
✓Network	nginx-node-redis_default	Removed	0.3

```
PS C:\Users\Xue Zhengyang\nginx-node-redis>
```

Containers [Give feedback](#)

Container CPU usage

0.06% / 1600% (16 CPUs available)

Container memory usage

93.66MB / 7.44GB

[Show charts](#)

☒ Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	>	● gitea-docker	-	-	-	0.06%	21 seconds ago	☐ ☐ ☐ ☐ ☐

The screenshot shows the Docker Desktop application. The top bar includes the Docker logo, the project name 'gitea-docker', and system status icons. The main area is divided into three panes. The left pane shows a tree view of the project structure, including 'server', 'gitea', and 'data'. The middle pane displays a log stream for the 'gitea' service, showing the initialization of various components like the database, repository, and indexers. The right pane shows the 'Multi-container applications' section with a list of containers and their status. The bottom status bar indicates system resources like RAM and CPU usage.

Images / docker.gitea.com/gitea:1.26.1

docker...

IN USE

CREATED

4 days ago

SIZE

244.96 MB

Recommended fixes

Run

Layers (12)

0	ADD alpine-minirootfs...	9.11 MB
1	CMD ["/bin/sh"]	0 B
2	EXPOSE [22/tcp 3000...	0 B
3	RUN /bin/sh -c apk --n...	48.17 MB
4	RUN /bin/sh -c addgr...	32.77 KB
5	COPY /tmp/local / # ...	98.3 KB
6	COPY /go/src/code.gi...	118.24 MB
7	ENV USER=gitea	0 B
8	ENV GITEA_CUSTOM...	0 B

Analyzed by

docker scout

Vulnerabilities

Packages

Give feedback

This image has not been analyzed

You can use Docker Scout to analyze local images and list its vulnerabilities.

Start analysis

Enable background indexing in Settings so your results are always ready.

Learning center

Multi-container applications

Estimated time: 8 mins

1

Introducing Docker Compose

If you've already completed the [How do I run a container?](#) guide, you saw that you must start each container individually. Imagine how great it would be if a tool could start multiple containers with a single command. That tool is Docker Compose.

Back

Next

2

Get the sample application

3

Understanding the sample

4

Digging into the compose.yaml

5

Running the application

RAM 2.43 GB CPU 0.25% Disk: 1.85 GB used (limit 1006.85 GB)

v4.66.1

```
PS C:\WINDOWS\system32> docker run -d -p 8080:80 docker/welcome-to-docker
Unable to find image 'docker/welcome-to-docker:latest' locally
latest: Pulling from docker/welcome-to-docker
fd372c3c84a2: Pull complete
958a74d6a238: Pull complete
9745203f5d34: Pull complete
828fa206d77b: Pull complete
bdaad27fd04a: Pull complete
a5585638209e: Pull complete
9824c27679d3: Pull complete
c1d2dc189e38: Pull complete
1babe895ddff: Download complete
Digest: sha256:c4d56c24da4f009ecf8352146b43497fe78953edb4c679b841732beb97e588b0
Status: Downloaded newer image for docker/welcome-to-docker:latest
4ff5997dff2eb8bf0559ea6049c3db9ff82718fb3d2b6c02a8a48922118d732d
PS C:\WINDOWS\system32>

PS C:\WINDOWS\system32> docker --version
Docker version 29.3.1, build c2be9cc
PS C:\WINDOWS\system32> docker run -d -p 8080:80 docker/welcome-to-docker
Unable to find image 'docker/welcome-to-docker:latest' locally
latest: Pulling from docker/welcome-to-docker
fd372c3c84a2: Pull complete
958a74d6a238: Pull complete
9745203f5d34: Pull complete
828fa206d77b: Pull complete
bdaad27fd04a: Pull complete
a5585638209e: Pull complete
9824c27679d3: Pull complete
c1d2dc189e38: Pull complete
1babe895ddff: Download complete
Digest: sha256:c4d56c24da4f009ecf8352146b43497fe78953edb4c679b841732beb97e588b0
Status: Downloaded newer image for docker/welcome-to-docker:latest
4ff5997dff2eb8bf0559ea6049c3db9ff82718fb3d2b6c02a8a48922118d732d
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS
4ff5997dff2e   docker/welcome-to-docker            "/docker-entrypoint..." 2 minutes ago   Up 2 minutes   0.0.0.0:8080->80
/tcp, [::]:8080->80/tcp
cba947608522   docker.gitea.com/gitea:1.26.1      "/usr/bin/entrypoint..." 2 days ago     Up 11 minutes   0.0.0.0:3000->30
00/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp
gitea
```

localhost:8080

Congratulations!!!

You ran your first container.

X L M

docker-demo

此电脑 > 本地磁盘 (C:) > 用户 > Xue Zhengyang > docker-demo

名称	修改日期	类型	大小
compose.yaml	2026/4/28 19:44	Yaml 源文件	1 KB

compose.yaml

```
C: > Users > Xue Zhengyang > docker-demo > compose.yaml
1 services:
2   app:
3     image: docker/welcome-to-docker
4     ports:
5       - 8080:80
```

```
PS C:\WINDOWS\system32> cd ~/docker-demo
PS C:\Users\Xue Zhengyang\docker-demo> docker compose up
[+] up 2/2
  ✓ Network docker-demo_default Created                                0.0s
  ✓ Container docker-demo-app-1 Created                                0.1s
Attaching to app-1
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint docker-demo-app-1 (die66967bde32ead7cf410c2c2dbadac7f505e7e300c5f8e01769f55146fcb94): Bind for 0.0.0.0:8080 failed: port is already allocated
PS C:\Users\Xue Zhengyang\docker-demo> docker compose up -d
[+] up 1/1
  ✓ Container docker-demo-app-1 Started                                0.1s
PS C:\Users\Xue Zhengyang\docker-demo> docker compose down
[+] down 2/2
  ✓ Container docker-demo-app-1 Removed                                0.2s
  ✓ Network docker-demo_default Removed                                0.3s
PS C:\Users\Xue Zhengyang\docker-demo>
```


docker desktop
PERSONAL
Search
Ctrl+K
?
🔔
📧
⚙️
⋮
X
—
□
×

Containers

Give feedback

Container CPU usage

0.23% / 1600% (16 CPUs available)

Container memory usage

152.19MB / 7.44GB

Show charts

Only show running containers

	Name	Container ID	Image	Port(s)	Actions
	suspicious_goo	4ff5997dff2e	docker/web	8080:80	
	determined_lan	7a0926aac86f	postgres	5432:5432	
	objective_hodgl	1f59d3a7cdc5	postgres	5433:5432	
	gitea-docker	-	-	-	

RAM 1.35 GB CPU 0.19% Disk: 2.55 GB used (limit 1006.85 GB)

Showing 4 Items

Learning center

Multi-container applications

Estimated time: 8 mins

1

Introducing Docker Compose

If you've already completed the [How do I run a container?](#) guide, you saw that you must start each container individually. Imagine how great it would be if a tool could start multiple containers with a single command. That tool is Docker Compose.

Back

Next

2

Get the sample application

3

Understanding the sample

4

Digging into the compose.yml

5

Running the application

```

PS C:\WINDOWS\system32> docker network create mynetwork
9aad5e935cd84b0f733efe53bc17d1dd9e11c159ef1e811a124117b808c93e9b
PS C:\WINDOWS\system32> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
de8cce4952ec        bridge             bridge              local
f78e4d8aff14        gitea-docker_gitea bridge              local
42e5ca8dfafb        host               host                local
9aad5e935cd8        mynetwork          bridge              local
50c63a8747a8        none               null                local
PS C:\WINDOWS\system32> docker network inspect mynetwork
[
  {
    "Name": "mynetwork",
    "Id": "9aad5e935cd84b0f733efe53bc17d1dd9e11c159ef1e811a124117b808c93e9b",
    "Created": "2026-04-28T12:08:19.240945155Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.19.0.0/16": {
            "IPsInUse": 3,
            "DynamicIPsAvailable": 65533
          }
        }
      }
    }
  }
]

```

```
PS C:\WINDOWS\system32> docker run -d -e POSTGRES_PASSWORD=secret -p 5434:5432 --network mynetwork postgres
219046d5627938c104e929d159c0e8d98fa4edfddcddec8e42f5430bff306168
```

```
PS C:\WINDOWS\system32> docker network inspect mynetwork
```

```
[
  {
    "Name": "mynetwork",
    "Id": "9aad5e935cd84b0f733efe53bc17d1dd9e11c159ef1e811a124117b808c93e9b",
    "Created": "2026-04-28T12:08:19.240945155Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {
      "219046d5627938c104e929d159c0e8d98fa4edfddcddec8e42f5430bff306168": {
        "Name": "loving_satoshi",
        "EndpointID": "5ab8759d5061a42b8ae70bd832aa12a06ee835ad405b9dd4b92cb32237c989f3",
        "MacAddress": "e2:8a:db:1a:ea:e8",
        "IPv4Address": "172.19.0.2/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.19.0.0/16": {

```

```
PS C:\WINDOWS\system32> docker run -d -e POSTGRES_PASSWORD=secret --memory="512m" --cpus=".5" postgres
cc29e837e0487c4515ed9b77aa3495e14408391fb75a6514b186104d895ddbba0
PS C:\WINDOWS\system32> mkdir ~/postgres-compose
```

目录: C:\Users\Xue Zhengyang

Mode	LastWriteTime	Length	Name
d-----	2026/4/28 20:11		postgres-compose

```
PS C:\WINDOWS\system32> cd ~/postgres-compose
```

compose.yml

C:\Users\Xue Zhengyang\postgres-compose> compose.yml

```
1 services:
2   postgres:
3     image: postgres:18
4     entrypoint: ["docker-entrypoint.sh", "postgres"]
5     command: ["-h", "localhost", "-p", "5432"]
6     environment:
7       POSTGRES_PASSWORD: secret
```

```
PS C:\Users\Xue Zhengyang\postgres-compose> docker compose up -d
[+] up 3/3
```

```
✓ Image postgres:18 Pulled
✓ Network postgres-compose_default Created
✓ Container postgres-compose-postgres-1 Started
```

```
PS C:\Users\Xue Zhengyang\postgres-compose> |
```

postgres-compose-
postgres-1

604aebd51052 postgres:11

STATUS
Running (59 seconds ago)

LogsInspectBind mountsExecFilesStateDebug modeOpen in external terminal

psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=#

PS C:\Users\Xue Zhengyang\postgres-compose> docker run -e POSTGRES_PASSWORD=secret postgres docker-entrypoint.sh -h localhost -p 5432
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are enabled.

fixing permissions on existing directory /var/lib/postgresql/18/docker ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default "max_connections" ... 100
selecting default "shared_buffers" ... 128MB
selecting default time zone ... Etc/UTC
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

Success. You can now start the database server using:

pg_ctl -D /var/lib/postgresql/18/docker -l logfile start

initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
waiting for server to start....2026-04-28 12:18:36.182 UTC [53] LOG: starting PostgreSQL 18.3 (Debian 18.3-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2026-04-28 12:18:36.186 UTC [53] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-04-28 12:18:36.197 UTC [59] LOG: database system was shut down at 2026-04-28 12:18:35 UTC
2026-04-28 12:18:36.202 UTC [53] LOG: database system is ready to accept connections
done
server started

/usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*

waiting for server to shut down...2026-04-28 12:18:36.296 UTC [53] LOG: received fast shutdown request
2026-04-28 12:18:36.303 UTC [53] LOG: aborting any active transactions
2026-04-28 12:18:36.304 UTC [53] LOG: background worker "logical replication launcher" (PID 62) exited with exit code 1
2026-04-28 12:18:36.305 UTC [57] LOG: shutting down

Success. You can now start the database server using:

```
pg_ctl -D /var/lib/postgresql/18/docker -l logfile start
```

```
initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
waiting for server to start....2026-04-28 12:18:36.182 UTC [53] LOG:  starting PostgreSQL 18.3 (Debian 18.3-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2026-04-28 12:18:36.186 UTC [53] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-04-28 12:18:36.197 UTC [59] LOG:  database system was shut down at 2026-04-28 12:18:35 UTC
2026-04-28 12:18:36.202 UTC [53] LOG:  database system is ready to accept connections
done
server started
```

```
/usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*
```

```
waiting for server to shut down...2026-04-28 12:18:36.296 UTC [53] LOG:  received fast shutdown request
2026-04-28 12:18:36.303 UTC [53] LOG:  aborting any active transactions
2026-04-28 12:18:36.304 UTC [53] LOG:  background worker "logical replication launcher" (PID 62) exited with exit code 1
2026-04-28 12:18:36.305 UTC [57] LOG:  shutting down
2026-04-28 12:18:36.309 UTC [57] LOG:  checkpoint starting: shutdown immediate
2026-04-28 12:18:36.334 UTC [57] LOG:  checkpoint complete: wrote 0 buffers (0.0%), wrote 3 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.007 s, sync=0.005 s, total=0.029 s; sync files=2, longest=0.003 s, average=0.003 s; distance=0 kB, estimate=0 kB; lsn=0/175F8E0, redo lsn=0/175F8E0
2026-04-28 12:18:36.348 UTC [53] LOG:  database system is shut down
done
server stopped
```

PostgreSQL init process complete; ready for start up.

```
2026-04-28 12:18:36.417 UTC [1] LOG:  starting PostgreSQL 18.3 (Debian 18.3-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2026-04-28 12:18:36.418 UTC [1] LOG:  listening on IPv6 address ":::1", port 5432
2026-04-28 12:18:36.418 UTC [1] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2026-04-28 12:18:36.425 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-04-28 12:18:36.434 UTC [73] LOG:  database system was shut down at 2026-04-28 12:18:36 UTC
2026-04-28 12:18:36.440 UTC [1] LOG:  database system is ready to accept connections
```

```
PS C:\WINDOWS\system32> docker run --name=db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/postgresql postgres:18
05d6d6d22684add726300ba5a057f35f9f04aed1be436311d03d2ee8c9b005ed
PS C:\WINDOWS\system32> docker exec -ti db psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.
```

```
postgres=# CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    description VARCHAR(100)
);
INSERT INTO tasks (description) VALUES ('Finish work'), ('Have fun');
CREATE TABLE
INSERT 0 2
postgres=# SELECT * FROM tasks;
 id | description
----+-----
  1 | Finish work
  2 | Have fun
(2 rows)
```

```
postgres=#
```

```
postgres=# \q
PS C:\WINDOWS\system32> docker stop db
db
PS C:\WINDOWS\system32> docker rm db
db
PS C:\WINDOWS\system32> docker run --name=new-db -d -v postgres_data:/var/lib/postgresql postgres:18
aac3ba778a62d73a622ff3c9ad0edf844740b2cd18bec3ca5142b876f6f67f25
```

```
PS C:\WINDOWS\system32> docker exec -ti new-db psql -U postgres -c "SELECT * FROM tasks"
 id | description
----+-----
  1 | Finish work
  2 | Have fun
(2 rows)
```

```
PS C:\WINDOWS\system32> |
```

Volumes [Give feedback](#)

		Search				Create
☐		Name ↑	Created	Size	Actions	
☐		5286b2140e195ffb1724577f1b2be84d47ff85c2832d4498af843e0f	4 days ago	39.1 MB		
☐		62379109fa6c3bcd9e26d28be4098d404100baca19f3591a126915	4 days ago	39.1 MB		
☐		66932964bd5ee2a42c13854ef44448685dfb8f87a263f70a6f5e2cd	4 days ago	39.1 MB		
☐		6f3a5dfe9a145a0ff9bb146ce7c26e6b05e44e6edc95419a946284e	4 days ago	39.1 MB		
☐		8f3762089f6e09947984abd2d4f8470c84c2f8cf0e561c22482d3afe	4 days ago	39.1 MB		
☐		ecf8d577ab260fe0c0efbe46547681daf88f266c438b91ce538eb1ce	4 days ago	39.1 MB		
☐		postgres_data	2 minutes ago	38.9 MB		

Showing 7 items

postgres_data [🔗](#)
● In use

Created
3 minutes ago



Import



Stored data Container in-use Exports

Name ↑	Size	Last modified	Mode
18	38.9 MB	3 minutes ago	drwxr-xr-x
docker > base > global > pg_commit_ts > pg_dynshmem pg_hba.conf pg_ident.conf > pg_logical > pg_multixact > pg_notify > pg_replslot	38.9 MB	2 minutes ago	drwx---
base	22.3 MB	3 minutes ago	drwx---
global	548.4 kB	1 second ago	drwx---
pg_commit_ts	0 Bytes	3 minutes ago	drwx---
pg_dynshmem	0 Bytes	3 minutes ago	drwx---
pg_hba.conf	5.6 kB	3 minutes ago	-rw---
pg_ident.conf	2.6 kB	3 minutes ago	-rw---
pg_logical	8 Bytes	2 minutes ago	drwx---
pg_multixact	16 kB	3 minutes ago	drwx---
pg_notify	0 Bytes	3 minutes ago	drwx---
pg_replslot	0 Bytes	2 minutes ago	drwx---

```
PS C:\WINDOWS\system32> docker rm -f new-db
new-db
PS C:\WINDOWS\system32> docker volume rm postgres_data
postgres_data
PS C:\WINDOWS\system32> docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
PS C:\WINDOWS\system32> |
```

```
PS C:\WINDOWS\system32> docker run -d -p 8080:80 --name my_site httpd:2.4
Unable to find image 'httpd:2.4' locally
2.4: Pulling from library/httpd
1935f25c7dc4: Pull complete
d32ec5ee95cb: Pull complete
4f4fb700ef54: Pull complete
b267b545226b: Pull complete
5b16784cbeed: Pull complete
555e17187742: Download complete
2a2591e319bc: Download complete
Digest: sha256:89a0b59e6d7285f00fc8df952a1579c31eb035cbdcc96690051e8bb6432cbea
Status: Downloaded newer image for httpd:2.4
679a9764aae7ea4567925ac639b05987554f9e4665960ab91d8be17dc676c62e
```



It works!

Containers [Give feedback](#)

Container CPU usage ⓘ **0.12%** / 1600% (16 CPUs available) Container memory usage ⓘ **93.82MB** / 7.44GB [Show charts](#)

Search ☐ Only show running containers [Delete](#)

		Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
☐	☐	cool_ardinghelli	cc29e837e048	postgres		0%	4 d	
☐	☐	loving_satoshi	219046d56279	postgres	5434:5432	0%	4 d	
☐	☐	objective_hodgk	1f59d3a7cdc5	postgres	5433:5432	0%	4 d	
☐	☐	determined_larr	7a0926aac86f	postgres	5432:5432	0%	4 d	
☐	☐	suspicious_goo	4ff5997dff2e	docker/wel	8080:80	0%	4 d	
☒	☒	my_site	679a9764aae7	httpd:2.4	8080:80	0%	2 m	
☐	☒	postgres-comp	-	-	-	0%	4 d	
☐	☒	gitea-docker	-	-	-	0.15%	14 r	

Selected 1 of 9

```
PS C:\Users\Xue Zhengyang> mkdir public_html
```

目录: C:\Users\Xue Zhengyang

Mode	LastWriteTime	Length	Name
d-----	2026/5/2 10:59		public_html

```
PS C:\Users\Xue Zhengyang> cd public_html
PS C:\Users\Xue Zhengyang\public_html>
```